

BUSINESS REQUIREMENTS DOCUMENT & SALES PROPOSAL

Project: GoKirana (Phase 1)

Version: 1.0 | Date: June 5, 2026

Target Audience: GoKirana Stakeholders / Investors & Clients

1. Executive Summary & Sales Pitch

The Kirana Revolution: Bridging Trust with Tech

In a rapidly digitizing economy, the local 'Kirana' shop remains the bedrock of Indian retail, built on years of personal relationships, immediate accessibility, and local trust. However, they face fierce competition from hyper-local quick-commerce giants.

GoKirana is designed to level the playing field. It is not just another app; it is a hyper-local ecosystem that empowers traditional Kirana store owners to go digital in minutes, while offering customers the familiarity of their local trusted shops combined with the convenience of modern e-commerce.

Why GoKirana Phase 1 is a Game Changer:

- **Unified Hyper-Local Experience:** A single, consolidated catalog displaying all products from nearby registered stores by category, offering a Blinkit/Zepto-like shopping experience without exposing individual shops or shop-wise listings.
- **Frictionless Onboarding & Security:** Fast registrations and secure logins via One-Time Passwords (OTP) or Passwords.
- **Instant Interactive Order Lifecycle:** A persistent acoustic alert system for shops that ensures zero missed orders, followed by an optimized 5-minute packing timeline and real-time transit tracking.
- **Secure Integrated Payments:** Out-of-the-box Razorpay integration to facilitate trust, digital-first refunds, and smooth checkout splits.
- **Universal Operations Control:** A central Admin Interface providing complete data observability, order audits, user metrics, and shop verification.

2. Project Architecture & Technical Stack

To ensure rapid deployment in Phase 1, followed by a seamless transition to enterprise scale, we have designed a robust, cost-effective tech stack combining React.js frontend clients with a high-performance Python API backend and a migrate-ready SQLite database tier.

Technical Component Breakdown

1. Frontend (React.js):

State-of-the-art Single Page Application (SPA) architecture. Core State Management (Redux Toolkit or React Context API) to handle cart, user sessions, and persistent web socket notification states. Styled using high-fidelity Tailwind CSS or premium Vanilla CSS modules to guarantee modern, ultra-responsive layouts.

2. Backend (Python):

Engineered using FastAPI or Flask for high-performance, asynchronous endpoints (essential for real-time WebSocket connection handling for shop alerts). Python-based background tasks to manage order expiration timers and callback loops.

3. Database (SQLite to PostgreSQL/SQL Server Migration Path):

Phase 1 Development: SQLite is utilized for seamless configuration, low memory footprint, and fast iterations.

Database Architecture Rules:

- GoKirana Shops Table: The master directory storing shop configurations, legal details, registration credentials, operating statuses, and coordinates.
- Individual Shop Tables/Dynamic Schema: To decouple shop data, each shop's product catalogs and inventory metrics will map to dynamically created shop-specific tables, or clean foreign-key partition logic.
- Unified Product Aggregation: While shops register and manage their catalogs individually, the system aggregates active inventories dynamically to render a single consolidated catalog to the customer.

Migration Ready: Using an ORM like SQLAlchemy ensures database schemas are entirely database-agnostic. Migrating from SQLite to SQL Server/PostgreSQL will require a simple connection-string update and migration execution.

3. Detailed Interface Feature Set

Interface 1: Shop Interface (The Merchant Hub)

Empowering shop owners with simple, powerful digital inventory and fulfillment tools.

- **Merchant Registration & Profile Setup:** Fast onboarding requiring mobile validation, shop name, address details, and store photos. Automatic directory entry into the core GoKirana Shops database.
- **Smart Catalog & Inventory Manager:** Create, update, or hide products. Input pricing, discount rates, and unit stock availability. Upload high-definition product images (stored in secure cloud storage).
- **Fulfillment Panel & Alert System:** Acoustic Alert Loop: When an order is placed, an auditory alert rings continuously on the merchant web app. This sound cannot be muted or dismissed until the merchant clicks Accept or Decline. The 5-Min Prep Timer: Upon order acceptance, a prominent, dynamic countdown timer of 5:00 minutes displays, guiding the shopkeeper through packing efficiency. Transition Trigger: Once packed, clicking 'Packed' shifts the order status instantly to 'In Transit' and updates the customer portal.

Interface 2: Customer Interface (The Shopping Portal)

A seamless, secure, and modern shopping experience.

- **Frictionless Dual Authentication:** Secure login using a combination of registered phone number and password. Passwordless OTP Login: Send a verified One-Time Password to the user's phone via SMS for quick access.
- **Single-Page Unified Catalog:** Displays a single unified shopping feed aggregating products from all registered local stores. Customers do not see individual shops or shop-wise listings (similar to Blinkit or Zepto).
- **Category-Driven Aggregated Browsing:** Displays all categories on the main page (e.g., Kitchenware, Grocery, Healthcare, Vegetables, Breads, etc.). Each category lists individual items in a card format containing:
 - Image of the item
 - Item name
 - Item description
 - MRP Price (if applicable)
 - Offered Price (our offering price)
 - 'Add to Cart' button
- **Cart, Checkout, & Razorpay Payment Gateway:** Interactive cart management. Selecting an item card opens the Item Specification Page (showing all available specifications). Payment gateway integrations (Razorpay) process checkout, routing orders to appropriate shops based on inventory allocation algorithms.
- **Order Tracking & Notifications:** Real-time order timeline: Ordered -> Preparing (5m timer) -> In Transit -> Delivered. Instant notification banner upon delivery completion.

Interface 3: Admin Interface (The Command Center)

A comprehensive panel for global platform operators.

- **Central Dashboard:** View overall platform health: Total active shops, registered customers, daily sales volume, and success rates.
- **Shop & Customer Audit Log:** Detailed listing of all shops (address, phone, validation status, catalog count) and customer profiles (phone, registration date, delivery address history).
- **Real-time Order Monitor:** Audit trails of all active and historic transactions with deep-dive payment receipts (Razorpay transaction IDs).

4. Third-Party API Integrations & Cost Estimates

To run GoKirana Phase 1, the following external cloud integrations are necessary. Below are estimated charges based on standard developer tier pricing.

Integration Category	Provider	Purpose	Cost Structure (Estimated Tier)
Payment Gateway	Razorpay	Processing UPI, credit cards, debit cards, and wallets.	No setup fee. Standard transaction fee: 2% per transaction (plus GST).
SMS Gateway	Twilio / Msg91	Sending OTPs during signup/login and critical order updates.	~₹0.15 to ₹0.25 per SMS (domestic India) or standard Twilio rates.
Cloud Media Storage	Amazon S3 / Cloudinary	Hosting shop product images.	Free tier (up to 10GB/month) or approx. ₹1,000/month for basic bandwidth.
Hosting (Phase 1)	AWS / Render	Backend API hosting & SQLite persistence.	Free/Developer Tiers available: Approx. ₹1,500/month for a VM.

5. Phase 1 Implementation & Verification Plan

Milestone Breakdown

Week 1: Schema Design & Backend Foundations

Establish Python project boilerplate. Setup SQLAlchemy models for GoKirana Shops, dynamic shop inventory structures, and the aggregated catalog query engine. Integrate JWT & Phone OTP verification loops.

Week 2: Front-end Interface Layouts & State Management

Build boilerplate for customer, shop, and admin React interfaces. Implement the unified single-page catalog customer flow (aggregated categories and product grids, item specification detail pages, cart/checkout state management) and admin/shop portals.

Week 3: Payment Checkout & Audio Alert Logic

Embed Razorpay client components. Implement WebSockets on backend and frontend to drive the persistent audio notification framework.

Week 4: End-to-End Testing & Migration Runbook

Manual end-to-end sandbox purchase verification. SQLite data load testing and SQL Server schema conversion verification.

Verification Strategy

- **Automated Testing:** Python unit testing suite (pytest) mocking payment callbacks, and React component test libraries verifying cart totals.
- **Manual Checkout Validation:** Executing a complete client flow in Razorpay's Sandbox mode to observe the shop UI response, audio alert interruption, and timer completion.